

Problem Set 3

BM2043 - Algorithms and Data Structure Lab

September 24, 2024

Implementation of Queue and Priority Queue

1. Implementation of Deque (Without STL)

Objective: Implement a deque (double-ended queue) data structure from scratch without using the Standard Template Library (STL). Your deque class should support basic deque operations: `pushFront`, `pushBack`, `popFront`, `popBack`, `front`, `back`, `isEmpty`, and `size`.

Instructions: Define a `Deque` class with the following methods:

- `pushFront(int value)`: Adds an integer `value` to the front of the deque.
- `pushBack(int value)`: Adds an integer `value` to the back of the deque.
- `popFront()`: Removes the front element from the deque and returns it. If the deque is empty, throw an exception or handle the underflow condition.
- `popBack()`: Removes the back element from the deque and returns it. If the deque is empty, throw an exception or handle the underflow condition.
- `front()`: Returns the front element of the deque without removing it. If the deque is empty, throw an exception or handle the underflow condition.
- `back()`: Returns the back element of the deque without removing it. If the deque is empty, throw an exception or handle the underflow condition.
- `isEmpty()`: Returns `true` if the deque is empty, otherwise `false`.
- `size()`: Returns the deque size.

Implement the deque using a circular buffer (fixed-size array) to store the elements. Include a constructor to initialize the deque with a given capacity. Handle edge cases such as attempting to pop from an empty deque or pushing onto a full deque.

Class Definition:

```
class Deque {
private:
    // Data members to store dequeue elements and the current size, front, back
public:
    // Constructor
    // Destructor
    // Push front operation
    // Push back operation
    // Pop front operation
    // Pop back operation
```

```

    // Front operation
    // Back operation
    // Check if deque is empty
    // Size
};

```

2. Implementation of Max-Heap (Without STL)

Objective: Implement a Max-Heap data structure from scratch without using the Standard Template Library (STL). The heap should support basic operations: insert, removeMax, getMax, size, and isEmpty. Additionally, implement heapify and heap sort functions.

Instructions: Define a `MaxHeap` class with the following methods:

- **insert(int value):** Inserts a new integer value into the heap, maintaining the heap property.
- **removeMax():** Removes the largest element (root of the heap) and returns it. The heap property should be maintained after the removal.
- **getMax():** Returns the largest element (root of the heap) without removing it. If the heap is empty, throw an exception or handle the underflow condition.
- **isEmpty():** Returns `true` if the heap is empty, otherwise `false`.
- **size():** Returns the current size of the heap.
- **heapify():** Converts an arbitrary array into a max-heap.
- **heapSort(vector<int>& array):** Sorts an array using the heap sort algorithm, returning a sorted array.

The heap should be implemented using an array (or dynamic array) to store elements. It should maintain the heap property where each parent node is greater than its child nodes. Use helper functions like `heapifyUp()` to maintain the structure when inserting, and `heapifyDown()` to maintain the structure when removing the maximum element.

Class Definition:

```

class MaxHeap {
private:
    // Data members to store the heap elements and the current size

public:
    // Constructor to initialize the heap
    // Destructor (if needed)

    void insert(int value);           // Insert a value into the heap
    int removeMax();                  // Remove and return the maximum element
    int getMax();                     // Return the maximum element without removing it
    bool isEmpty();                   // Check if the heap is empty
    int size();                       // Return the current size of the heap
    void heapify();                    // Convert an arbitrary array into a max-heap
    void heapSort(vector<int>& array); // Sort an array using heap sort

private:
    void heapifyUp(int index);        // Helper function to maintain the heap property after insertion
    void heapifyDown(int index);      // Helper function to maintain the heap property after removal
};

```

1 Problems on Queues and Priority Queues

Problem Statement 1

Shweta loves laddo and wants the sweetness of some laddo to be greater than a given value k . To do this, two laddo with the least sweetness are repeatedly mixed. This creates a special combined laddo with sweetness calculated as:

$$\text{sweets} = (1 \times \text{Least sweet laddo} + 2 \times \text{2nd least sweet laddo})$$

This operation occurs until all laddo have a sweetness greater than or equal to k .

Task: Given the sweetness of a number of laddo, determine the minimum number of operations required. If it is not possible to reach the desired sweetness, return -1 .

Example: For $k = 9$ and $A = [2, 7, 3, 6, 4, 6]$:

1. The smallest values are 2, 3. Combine them to get $1 \times 2 + 2 \times 3 = 8$, resulting in $A = [8, 7, 6, 4, 6]$.
2. Remove 4 and 6, combine them to get $1 \times 4 + 2 \times 6 = 16$, resulting in $A = [16, 8, 7, 6]$.
3. Remove 6 and 7, combine them to get $1 \times 6 + 2 \times 7 = 20$, resulting in $A = [20, 16, 8]$.
4. Remove 8 and 7, combine them to get $1 \times 7 + 2 \times 8 = 23$, resulting in $A = [23, 20, 16]$.

The number of operations required is 4.

Constraints:

- $1 \leq n \leq 10^6$
- $0 \leq k \leq 10^9$
- $0 \leq A[i] \leq 10^6$

Sample Input:

```
n = 6
k = 7
A = [1, 2, 3, 9, 10, 12]
```

Expected Output:

2

Explanation of Sample Input: Combine the first two laddo with sweetness 1 and 2:

- New sweetness = $1 \times 1 + 2 \times 2 = 5$
- Array becomes: $[3, 5, 9, 10, 12]$

Combine the two smallest laddo (3 and 5):

- New sweetness = $1 \times 3 + 2 \times 5 = 13$
- Array becomes: $[9, 10, 12, 13]$

Now all laddo have a sweetness greater than or equal to 7, so the process stops. The minimum number of operations required is 2.

Class Template You can use the following class template to implement the solution in C++:

```
class Solution {
public:
    int laddo(int k, vector<int> A) {
        // implement your solution here
    }
};
```

Problem Statement 2: (Kth Largest Element in a Stream)

Nishant is working as a Teaching Assistant (TA) for Algorithms and Data Structure Lab and needs to implement a system to track the k th largest test score from applicants in real-time. This will assist in determining cut-off marks dynamically as new scores are submitted.

Class Definition: You need to implement the `KthLargest` class with the following methods:

- **Constructor:**

```
KthLargest(int k, int[] nums)
```

Initializes the object with the integer k and a stream of test scores *nums*.

- **Method:**

```
int add(int val)
```

Adds a new test score *val* to the stream and returns the k th largest element from the current scores.

Examples:

Example 1:

Input:

```
["KthLargest", "add", "add", "add", "add", "add"]
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]
```

Output: [null, 4, 5, 5, 8, 8]

Explanation:

- `KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);`
- `kthLargest.add(3);` // returns 4
- `kthLargest.add(5);` // returns 5
- `kthLargest.add(10);` // returns 5
- `kthLargest.add(9);` // returns 8
- `kthLargest.add(4);` // returns 8

Example 2:

Input:

```
["KthLargest", "add", "add", "add", "add"]  
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]
```

Output: [null, 7, 7, 7, 8]

Constraints:

- $0 \leq \text{nums.length} \leq 10^4$
- $1 \leq k \leq \text{nums.length} + 1$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $-10^4 \leq \text{val} \leq 10^4$
- At most 10^4 calls will be made to `add`.

Class Template:

```
/**  
 * Your KthLargest object will be instantiated and called as such:  
 * KthLargest* obj = new KthLargest(k, nums);  
 * int param_1 = obj->add(val);  
 */  
  
class KthLargest {  
public:  
    KthLargest(int k, vector<int>& nums) {  
        // Implement the constructor logic  
    }  
  
    int add(int val) {  
        // Implement the logic to add the new score and return kth largest  
    }  
};
```

This class will help manage test scores efficiently, providing real-time updates on the k th largest score.

Problem Statement 3: (Merge K Sorted Arrays)

Nisarg is tasked with merging k sorted arrays, which are arranged in the form of a matrix of size $k \times k$. The goal is to merge them into one sorted array and return the result.

Class Definition: You need to implement the `Solution` class with the following method:

1. Method:

- `vector<int> mergeKArrays(vector<vector<int>> arr, int K):` Merges k sorted arrays and returns the merged sorted array.

Examples:

1. Example 1:

Input: $k = 3$, $\text{arr}[][] = \{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$
Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Explanation: The merged list is [1, 2, 3, 4, 5, 6, 7, 8, 9].

2. Example 2:

Input: $k = 4$, $\text{arr}[][] = \{\{1,2,3,4\},\{2,2,3,4\},\{5,5,6,6\},\{7,8,9,9\}\}$
Output: [1, 2, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 8, 9, 9]
Explanation: The merged list is [1, 2, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 8, 9, 9].

Expected Complexity:

- **Time Complexity:** $O(k^2 \log k)$
- **Auxiliary Space:** $O(k^2)$

Constraints:

- $1 \leq k \leq 10$

Class Template You can use the following class template to implement the solution in C++:

```
class Solution {
public:
    // Function to merge k sorted arrays.
    vector<int> mergeKArrays(vector<vector<int>>> arr, int K) {
        // code here
    }
};
```

Problem Statement 4: (Median from Data Stream)

Abhishek wants to implement a `MedianFinder` class that can handle a stream of integers and provide the median efficiently.

Class Definition: You need to implement the `MedianFinder` class with the following methods:

1. Constructor:

- `MedianFinder()`: Initializes the `MedianFinder` object.

2. Methods:

- `void addNum(int num)`: Adds the integer `num` from the data stream to the data structure.
- `double findMedian()`: Returns the median of all elements added so far. Answers within 10^{-5} of the actual answer will be accepted.

Examples:

1. Example 1:

Input:
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output:
[null, null, null, 1.5, null, 2.0]

Explanation:
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1); // arr = [1]
medianFinder.addNum(2); // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3); // arr = [1, 2, 3]
medianFinder.findMedian(); // return 2.0

Constraints:

- $-10^5 \leq num \leq 10^5$
- There will be at least one element in the data structure before calling `findMedian`.
- At most 5×10^4 calls will be made to `addNum` and `findMedian`.

Class Template: You can use the following class template to implement the solution in C++:

```
/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder* obj = new MedianFinder();
 * obj->addNum(num);
 * double param_2 = obj->findMedian();
 */
class MedianFinder {
public:
    MedianFinder() {
        // Initialize data structures here
    }

    void addNum(int num) {
        // Code to add number to data structure
    }

    double findMedian() {
        // Code to calculate and return the median
    }
};
```

This implementation will allow Abhishek to efficiently manage a stream of numbers and calculate the median dynamically.

Problem Statement 5: (Top K Frequent Elements)

Arin is analyzing data and needs to identify the top k most frequent elements in a dataset represented by an integer array *nums*. The goal is to return these k elements, which can be in any order.

Examples:

1. Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Explanation: The number 1 appears most frequently, followed by 2.

2. Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Explanation: Since there's only one element, it is the most frequent.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow Up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the size of the array.

Class Definition: You need to implement the following class:

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        // Implement the logic to find the top k frequent elements
    }
};
```

This task requires implementing an efficient algorithm to find the k most frequent elements while adhering to the time complexity constraint.

Problem Statement 6: Replace Each Element with Its Rank

Objective: Abhay is tasked with processing an array of integers by replacing each element with its rank when sorted in ascending order. The rank is determined by the position of the first occurrence of each element in the sorted version of the array. If there are duplicates, they should all have the same rank as the first occurrence.

Examples:

Example 1:**• Input:**

```
N = 6
arr = [20, 15, 26, 2, 98, 6]
```

• Output:

```
[4, 3, 5, 1, 6, 2]
```

- **Explanation:** After sorting, the array becomes {2, 6, 15, 20, 26, 98}. The ranks are determined as follows:

- Rank(2) = 1
- Rank(6) = 2
- Rank(15) = 3
- Rank(20) = 4
- Rank(26) = 5
- Rank(98) = 6

Example 2:**• Input:**

```
N = 4
arr = [2, 2, 1, 6]
```

• Output:

```
[2, 2, 1, 3]
```

- **Explanation:** After sorting, the array becomes {1, 2, 2, 6}. The ranks are determined as follows:

- Rank(1) = 1
- Rank(2) = 2 (both occurrences of 2 share the same rank)
- Rank(6) = 3

Your Task: Complete the function `vector<int> replaceWithRank(vector<int> &arr, int N)` which takes an integer N and an array of N integers as input and returns a new array where each element is replaced by its rank.

Expected Time Complexity:

- $O(N \cdot \log N)$

Expected Auxiliary Space:

- $O(N)$

Constraints:

- $1 \leq N \leq 10^5$
- $1 \leq arr[i] \leq 10^9$

Class Definition:

```
class Solution {
public:
    vector<int> replaceWithRank(vector<int> &arr, int N) {
        // Implement the logic to replace each element by its rank
    }
};
```

Problem Statement 7: Task Scheduler

Objective: Nishant is working on a task scheduler that handles CPU tasks labeled from A to Z, with a requirement for a minimum gap between tasks of the same label. Given an array of tasks and a cooling interval n , the goal is to determine the minimum number of CPU intervals needed to complete all tasks.

Examples:**Example 1:**

- **Input:**

```
tasks = ["A", "A", "A", "B", "B", "B"]
n = 2
```

- **Output:**

8

- **Explanation:** A possible execution sequence is: A → B → idle → A → B → idle → A → B. After executing an A, you must wait two intervals before executing another A.

Example 2:

- **Input:**

```
tasks = ["A", "C", "A", "B", "D", "B"]
n = 1
```

- **Output:**

6

- **Explanation:** A possible execution sequence is: A → B → C → D → A → B.

Example 3:

- **Input:**

```
tasks = ["A", "A", "A", "B", "B", "B"]  
n = 3
```

- **Output:**

10

- **Explanation:** A possible execution sequence is: A → B → idle → idle → A → B → idle → idle → A → B. Idling is required to satisfy the cooling constraint.

Your Task: Complete the function `int leastInterval(vector<char>& tasks, int n)` which takes a vector of tasks and an integer n and returns the minimum number of intervals required to complete all tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 10^4$
- `tasks[i]` is an uppercase English letter.
- $0 \leq n \leq 100$

Class Definition:

```
class Solution {  
public:  
    int leastInterval(vector<char>& tasks, int n) {  
        // Implement the logic to calculate the least intervals needed  
    }  
};
```